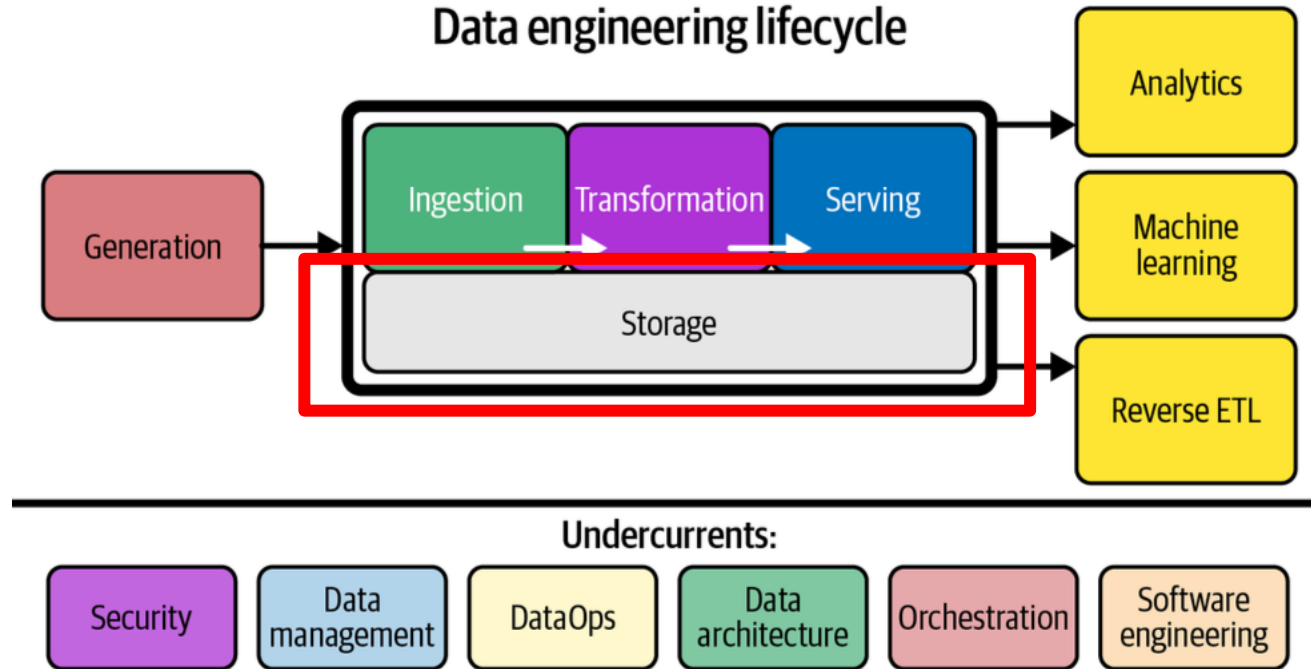


Data Engineering Storage

UA.DETI.ED - 2025/26

José Luís Oliveira

The Data Engineering Lifecycle



The Storage Spectrum: An Overview

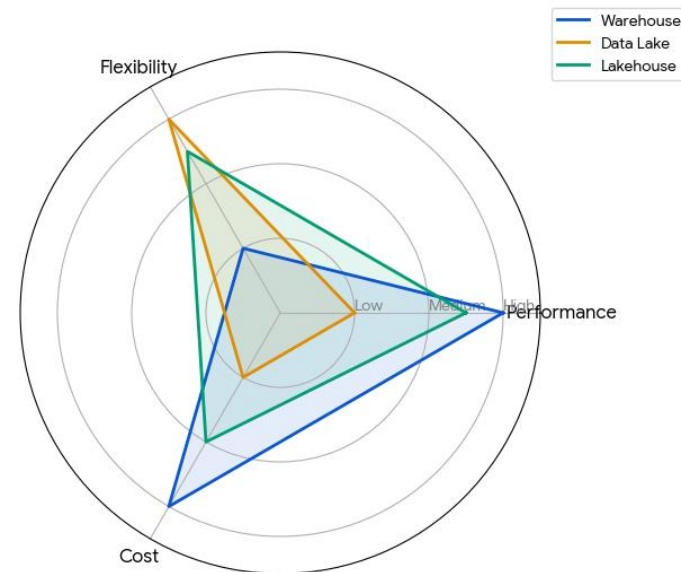
❖ Evolution of Data Management

- 1990s - Data Warehouse: Structured, SQL-only, expensive.
- 2010s - Data Lake: Unstructured, cheap, "Schema-on-Read."
- 2020s - Data Lakehouse: The convergence. Transactional guarantees (ACID) on cheap object storage.

❖ The Trade-off Triangle -

Cost vs. Performance vs. Flexibility:

- Warehouse: High Perf, Low Flexibility, High Cost.
- Lake: Low Performance, High Flexibility, Low Cost.
- Lakehouse: Balances all three using Table Formats.



Data Warehouse

❖ Theoretical Foundations of Data Warehousing

- Subject-Oriented: Data organized by major business entities (Customer, Product, Sales) rather than by application silos.
- Integrated: Inconsistent data from disparate sources is homogenized into a single, coherent structure.
- Non-Volatile: Data is loaded typically in bulk and rarely changed, creating an immutable historical record.
- Time-Variant: Every record contains a time element, enabling trend analysis over history.

❖ The Fundamental Shift: OLTP vs. OLAP

- OLTP (Operational): Optimized for fast, atomic row-level transactions (e.g., processing a payment).
- OLAP (Analytical): **The Warehouse paradigm**. Optimized for complex aggregation queries across millions of records (e.g., "Sum sales by region for Q4").

Data Warehouse

❖ Architecture & Characteristics

- Schema-on-Write: You must define the table structure (DDL) before loading data. If the data doesn't match the schema, the load fails.
- Storage Engine: Proprietary, highly optimized formats (e.g., Snowflake micro-partitions, BigQuery Capacitor).
- Compute: Tightly coupled or managed compute. You pay a premium for the query engine.

❖ Primary Use Cases

- Enterprise Reporting & Business Intelligence (BI)
 - Serving as the backend for dashboards (Tableau, PowerBI) used by executives and managers for KPI tracking.
 - Regulatory and financial reporting where extreme data accuracy and governance are required.
- Ad-Hoc Strategic Analysis

Data Warehouse - Advantages and Disadvantages

❖ The Advantages: Governance and Performance

- Single Source of Truth: Rigorous integration forces data consistency, reducing business ambiguity.
- High Query Performance: Columnar storage engines deliver sub-second responses on petabytes of structured data.
- Business Usability: Dimensional models (Star Schemas) are easier for non-engineers to understand and query than raw normalized data.

❖ The Disadvantages: Cost and Agility

- High TCO (Total Cost of Ownership): Storage and compute in commercial DWs are significantly more expensive than Data Lakes.
- Rigidity (Schema-on-Write): The structure must be defined before loading data. Adapting to new data types requires significant engineering effort.
- Data Latency: The complex transformation (ETL) required to structure data introduces delays.

Data Warehouse - Solutions

❖ Commercial Cloud Leaders (SaaS)

- Snowflake: The pioneer of decoupled storage and compute, known for ease of use and zero-maintenance.
- Google BigQuery: A truly serverless, highly scalable DW that abstracts away almost all infrastructure management.
- AWS Redshift: The original cloud DW, now evolving with serverless options and data lake integration.
- Databricks SQL: A "Lakehouse" approach providing a DW-like SQL experience directly on top of Data Lake storage (Delta Lake).

❖ Open Source & High-Performance Alternatives

- ClickHouse: An open-source columnar DBMS known for blazing fast real-time analytics.
- Apache Druid: Specialized stores for high-concurrency, low-latency user-facing analytics.
- Trino (formerly PrestoSQL): A distributed SQL query engine that acts as a "Virtual Data Warehouse" over disparate sources.

Data Lake

The Data Lake Pattern

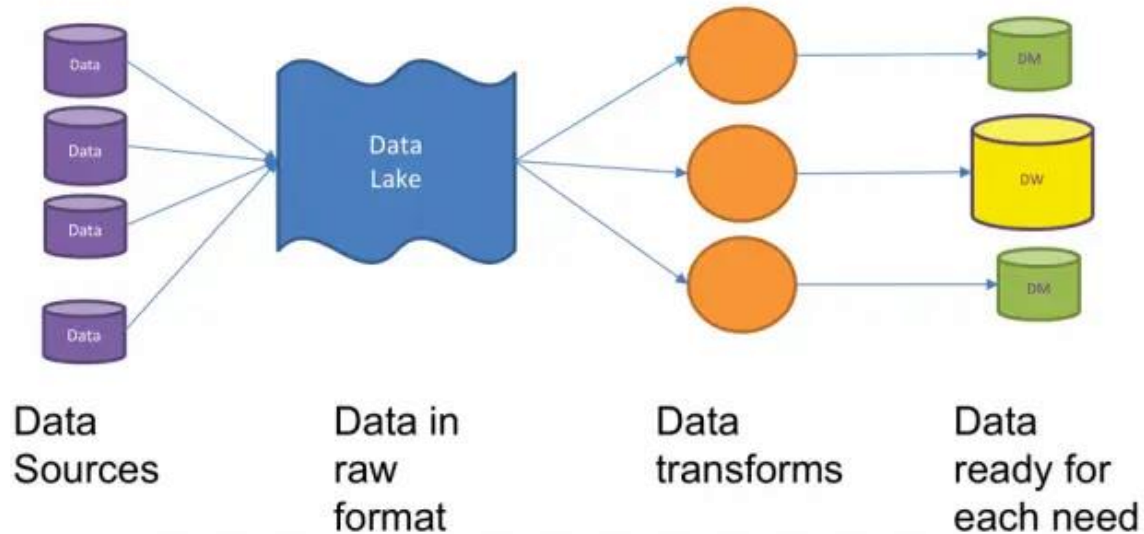


Image source: datakitchen.io

Data Lake

❖ Architecture & Characteristics

- Schema-on-Read: You dump raw files (JSON, CSV, Parquet) into folders. The schema is applied only when you query the data.
- Storage: Commodity Object Storage (Amazon S3, Azure ADLS, Google GCS). Extremely cheap (\$0.023/GB).
- Compute: Decoupled. You can use any engine (Spark, Presto, Python, DuckDB) to read the same files.

❖ Primary Use Cases

- Machine Learning: Training models on raw images or text.
- Archival: Storing 10 years of audit logs for compliance.
- Exploratory Analytics: Data Scientists digging into messy data to find value.

Data Lake

❖ The "Data Swamp" Problem

- Issue: Without schema enforcement, anyone can dump anything.
- Result: "Write-Once, Read-Never." Analysts can't figure out how to parse the data or trust its quality.
- Small Files: Streaming ingestion creates millions of tiny files, killing query performance.

❖ Real-World Example: Netflix

- Scale: Petabytes of data ingested daily.
- Strategy: Store everything in S3 (Lake). Use Spark to process and "Gold" tables for high-performance viewing.

Data Lake

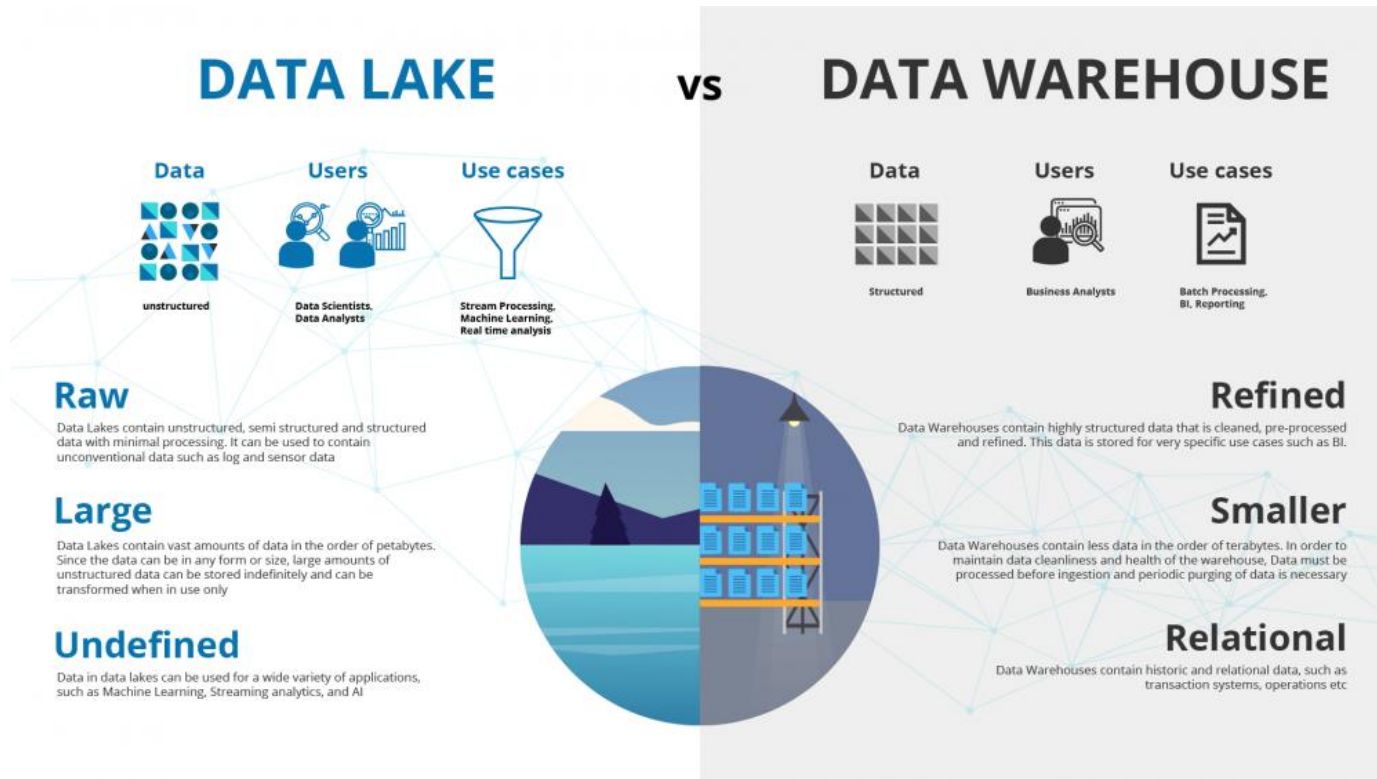
❖ File Formats Matter

- Parquet: Columnar. Best for analytics (Reading). Compressed.
- Avro: Row-based. Best for ingestion (Writing). Schema included.
- JSON/CSV: Only for landing zones. Convert immediately.

❖ The Missing Piece

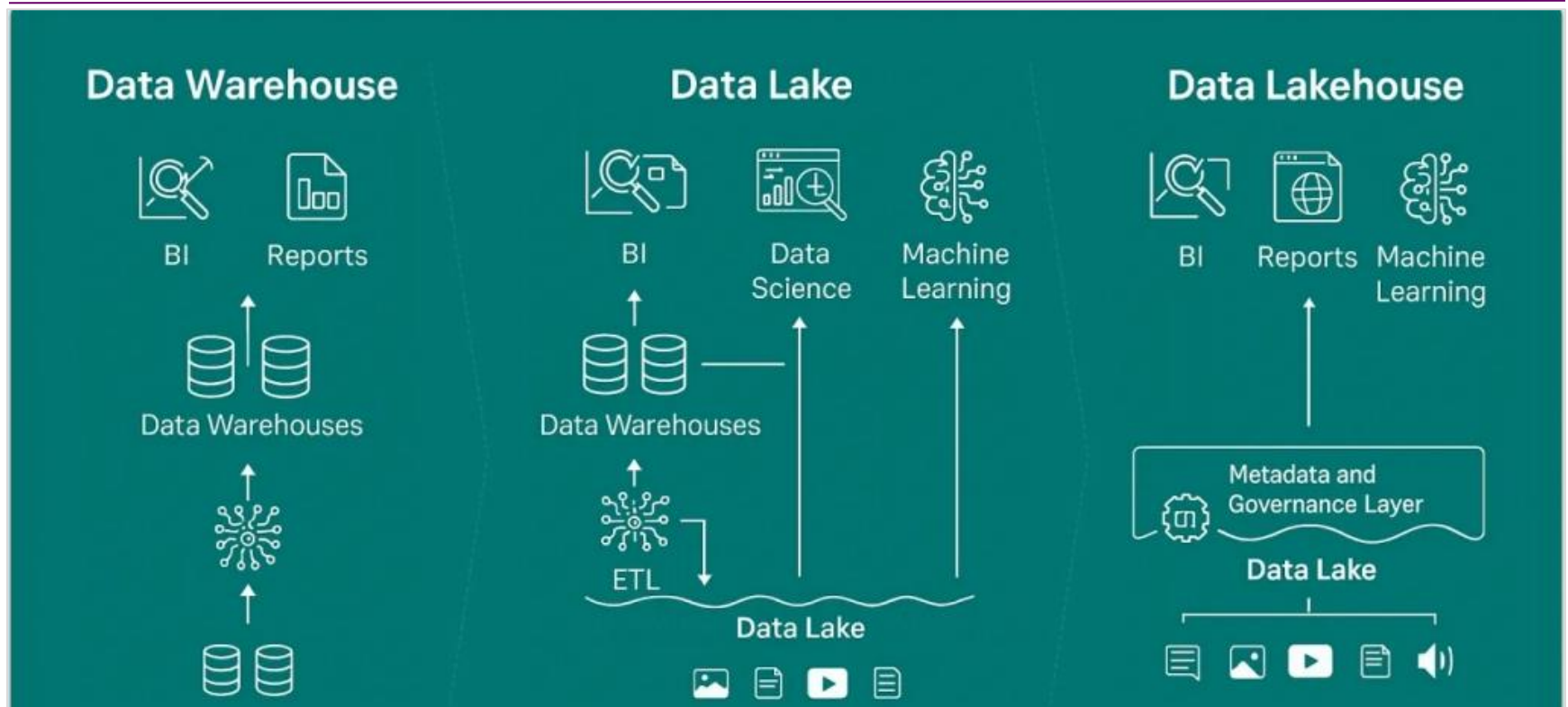
- Standard Data Lakes lack Transactions. If a job fails halfway through writing 1,000 files, users see partial data. There is no "Rollback."

Data Lake vs Data Warehouse



<https://www.qubole.com/data-lakes-vs-data-warehouses-the-co-existence-argument>

Data Lakehouse



<https://sranalytics.io/blog/data-lake-vs-data-warehouse-vs-data-lakehouse/>

Data Lakehouse

❖ The Convergence of Paradigms

– Definition

- A new data management architecture that implements data warehouse features (ACID transactions, schema enforcement) directly on low-cost data lake storage.

– The "Open" Philosophy

- Unlike traditional warehouses that lock data into proprietary formats, Lakehouses use open file formats (Parquet, Avro, ORC).

– Metadata Layers

- The "magic" isn't in the storage but in a new metadata layer (Table Formats) that tracks file versions, enabling features like Time Travel and atomic updates.

❖ Key Architectural Traits

– ACID Support

- Guarantees atomicity, consistency, isolation, and durability for concurrent reads and writes.

– Schema Enforcement

- Prevents "garbage in" by rejecting writes that don't match the table definition, solving the "Data Swamp" problem.

– Decoupled Compute & Storage

- Scales processing power independently of storage capacity.

Data Lakehouse - Use Cases

❖ Unified Analytics (BI + AI)

- Machine Learning - Data scientists can access the same raw data as business analysts without needing to copy it to a separate system.
- Business Intelligence - Running high-performance SQL queries on recent data for dashboards (Tableau/PowerBI) directly from the lake.

❖ Streaming and Real-Time Data

- Streaming Ingestion: Handling continuous inserts and updates (Upserts) from Kafka/CDC streams, which is difficult in traditional warehouses.
- Efficiently deleting specific user records from petabytes of data using transactional deletes (DELETE FROM table WHERE user_id = X).

❖ Data Quality & Governance

- Implementing "Bronze-Silver-Gold" quality layers within a single storage platform, simplifying lineage and access control.

Data Lakehouse – Pros and Cons

❖ The Advantages: Simplicity and Cost

- Eliminates Data Redundancy: No need to maintain two copies of data (one in the Lake for ML, one in the Warehouse for BI). "Store once, query everywhere."
- Lower cost: Leverages cheap object storage (S3/ADLS) for everything.
- Vendor Independence: Data is stored in open formats. we can swap the query engine (e.g., switch from Spark to Trino) without moving the data.

❖ The Disadvantages: Maturity and Complexity

- Management Overhead: Requires more engineering effort to configure and optimize (compaction, vacuuming) compared to "Zero-maintenance" SaaS warehouses.
- Tooling Maturity: The ecosystem (permissions, catalogs) is rapidly evolving but arguably less mature than established Enterprise Data Warehouses.

Data Lakehouse – Solutions

❖ The Open Source Table Formats (The "Big Three")

- Apache Iceberg: Born at Netflix. Focuses on huge scale, strict correctness, and engine neutrality. Widely adopted by AWS, Snowflake, and Apple.
- Delta Lake: Originally from Databricks. Highly optimized for Spark. Features "Z-Ordering" for performance. Now fully open governance (Linux Foundation).
- Apache Hudi: Born at Uber. optimized for streaming upserts and near-real-time ingestion pipelines.

❖ Commercial & Managed Platforms

- Databricks SQL: A commercial managed Lakehouse platform providing a warehouse-like experience on Delta Lake.
- Dremio: A "SQL Lakehouse" engine based on Apache Arrow, designed to query Iceberg/Delta data with sub-second latency.
- Snowflake (Hybrid): Now supports reading/writing Iceberg tables, effectively allowing it to act as a Lakehouse engine.

Data Lakehouse

❖ Deep Dive: Table Formats (Iceberg)

- Metadata Layer: Iceberg maintains a "Manifest List" (a list of all files in the table).
- Atomic Commits: To update the table, you write a new Manifest List. The pointer swaps instantly. Readers never see partial writes.
- Partition Evolution: You can change partitioning (e.g., from Month to Day) without rewriting old data.

❖ Deep Dive: Delta Lake

- Z-Ordering: A technique to co-locate related data in the same set of files, dramatically speeding up queries.
- Vacuum: A command to physically delete old file versions to save space (and comply with GDPR).

❖ Comparison

- Iceberg: Open governance (Apache). Best for interoperability (AWS, Netflix, Apple).
- Delta: Best performance on Databricks.

Data Mesh Architecture

- ❖ The **data as a product** principle - a product thinking philosophy onto analytical data.
 - The domain team is responsible for satisfying the needs of other domains by providing high-quality data. Basically, domain data should be treated as any other public API.
- ❖ The **domain ownership** principle mandates the domain teams to take responsibility for their data.
 - According to this principle, analytical data should be composed around domains, similar to the team boundaries aligning with the system's bounded context.
- ❖ A **self-serve data infrastructure platform**
 - A dedicated data platform team provides domain-agnostic functionality, tools, and systems to build, execute, and maintain interoperable data products for all domains.
- ❖ The **federated governance** principle achieves interoperability of all data products through standardization.
 - The main goal of federated governance is to create a data ecosystem with adherence to the organizational rules and industry regulations.

<https://www.datamesh-architecture.com/>

Data Mesh – A Decentralized Storage Pattern

❖ Concept

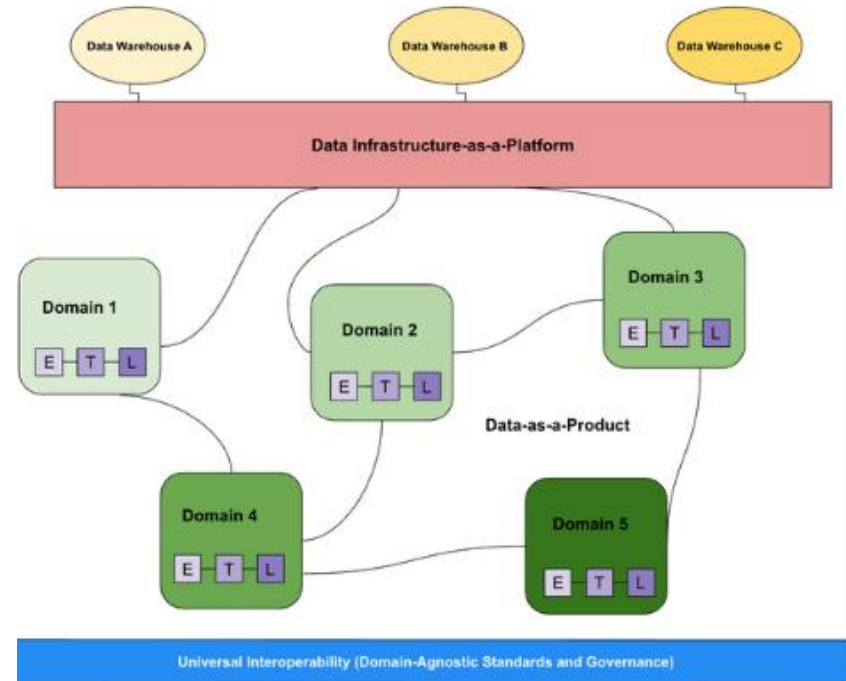
- Instead of one giant Monolithic Lake/Warehouse, break storage into "Domains" (Marketing, Sales, Product).

❖ Data as a Product

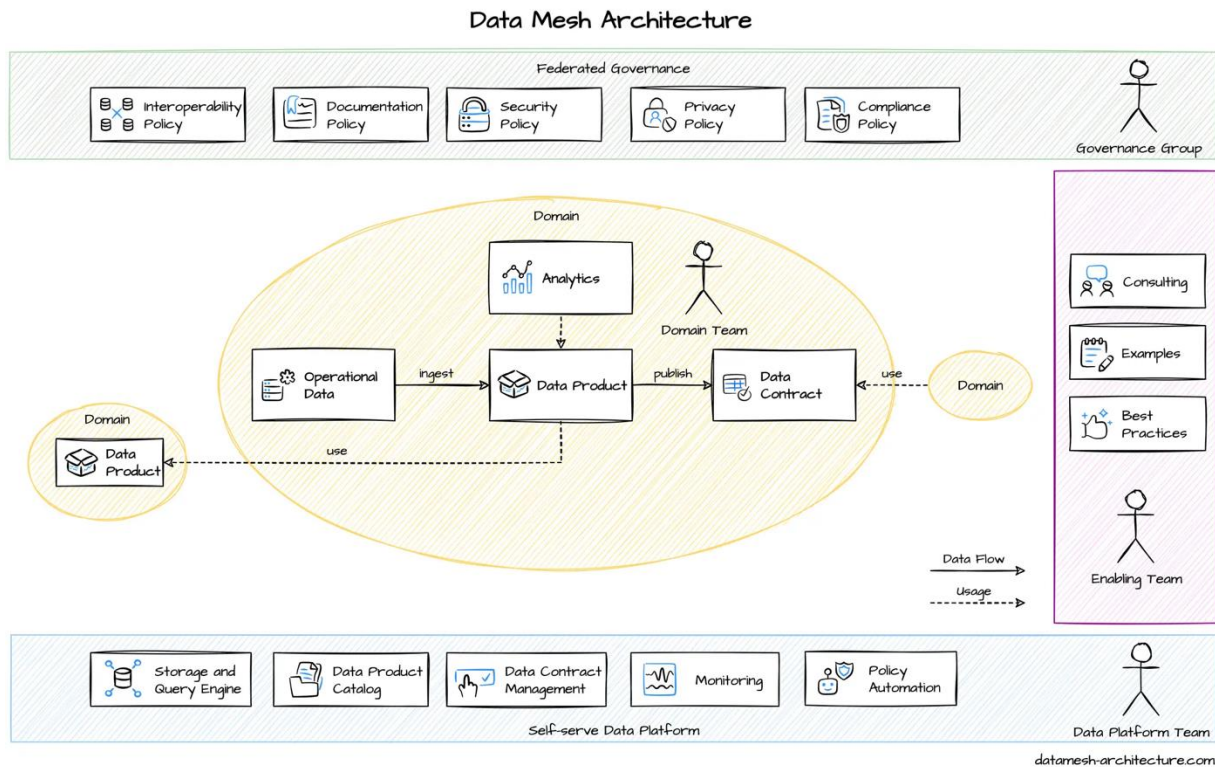
- Each domain owns their storage bucket and exposes "Data Products" (clean tables) to others.

❖ Governance

- A central catalog ensures standards, but storage is distributed.



Data Mesh Architecture



Data Mesh Architecture

❖ Storage Implementation

– Polyglot Storage

- The Marketing domain might use Snowflake (Warehouse). The IoT Product domain might use S3 + Iceberg (Lakehouse).

– Interoperability

- The key is standard interfaces (SQL, Iceberg) so domains can query each other without moving data.

❖ Real-World Example

- Challenge: Bottlenecked central data team.
- Shift: Moved to Data Mesh. Domain teams own their pipelines and storage.
- Result: Faster innovation, but requires strong governance tooling.

Data Virtualization - The "Zero Storage" Approach

- ❖ A data integration strategy that provides a unified, logical view of data from disparate sources without physically moving or copying the data.
 - "Leave the data where it is." The engine connects to the source (SQL DB, S3, API) at query time, fetches only the needed rows, and joins them in memory.
 - Unlike ETL (which copies data to a Warehouse), Virtualization acts as a translation layer between the user and the source systems.
- ❖ How Virtualization Works - Federated Query Architecture
 - Connectors:
 - The engine uses specialized connectors to "speak" the native language of the source (e.g., converting a SQL query into a MongoDB API call or an S3 range request).
 - Pushdown Optimization:
 - The engine pushes as much work as possible to the source system.
 - In-Memory Join:
 - Results from different sources (e.g., MySQL and S3) are pulled into the Virtualization cluster's memory, joined, and served to the user.

Data Virtualization - The "Zero Storage" Approach

❖ Primary Use Cases

– Ad-Hoc Analysis:

- "Join the CSV on my laptop with the Production DB to check a hypothesis." No need to build an ETL pipeline just for a one-off question.

– Logical Data Warehouse:

- Creating a "Virtual Schema" that combines data from a Legacy Oracle DW and a new Snowflake DW, allowing users to query them as if they were one system during a multi-year migration.

– Regulatory Compliance (GDPR):

- querying sensitive data across regions without physically moving it across borders (Data Sovereignty).

❖ Benefits and Trade-offs

– Pros:

- Instant Access: Zero data movement latency.
- Lower Storage Cost: No duplicate data storage.
- Agility: New data sources can be "connected" in minutes, not days.

– Cons:

- Performance: Queries are bounded by the speed of the *slowest* source system (e.g., querying a Salesforce API is always slow).
- Source Load: Heavy analytical queries can overwhelm the operational (OLTP) source database.

Storage Tiering Patterns

❖ Optimizing Cost vs. Access Speed

❖ Hot Tier (Standard)

- Designed for active data requiring millisecond latency and high throughput. Used for Ingestion, ETL scratchpads, and current quarter reporting.
 - Examples: AWS S3 Standard, Azure Blob Hot, Google Cloud Standard.
 - Cost Profile: Highest storage cost, lowest access cost.

❖ Warm Tier (Infrequent Access)

- Offers millisecond latency but charges significant fees for data retrieval. Ideal for data accessed monthly or yearly (e.g., Disaster Recovery backups, Year-over-Year analysis).
 - Examples: AWS S3 Standard-IA, Azure Blob Cool.

❖ Cold Tier (Archive):

- Extremely low storage cost but requires minutes to hours for retrieval. Used for long-term retention, audit logs, and compliance data that is legally required but rarely touched.
 - Examples: AWS Glacier Deep Archive, Azure Blob Archive.

Data Lifecycle Management (DLM)

❖ Automating Data Movement

- The set of automated rules that transition data between storage tiers based on object age, prefix, or access patterns.

❖ Intelligent Tiering

- A modern pattern where the cloud provider monitors access patterns and automatically moves objects to cheaper tiers when not accessed, without performance impact or retrieval fees (though often with a small monitoring fee).

❖ Time To Live (TTL)

- A critical configuration in streaming (Kafka/Kinesis) and caching (Redis) layers to auto-expire data, preventing storage backlogs and ensuring freshness.

❖ Policy Example

- Ingest to S3 Standard (Hot). If untouched for 30 days, move to Infrequent Access. After 365 days, move to Glacier Deep Archive. Hard delete after 7 years.

Storage Economics - The "Write Once, Pay Forever" Trap

❖ Problem:

- Storage costs are cumulative. Without lifecycle policies, data volume grows linearly or exponentially while its business value drops exponentially over time.

❖ Solution:

- Implement aggressive retention policies and "Delete by Default" for non-critical environments (e.g., Dev/Test environments should auto-wipe every 30 days).

❖ Cost Allocation:

- Enforcing strict Metadata Tagging (e.g., CostCenter: Marketing, Project: GenAI) on storage buckets is the only way to charge costs back to specific business units, incentivizing them to clean up their own data.

Managing Regulatory Deletion (GDPR/CCPA)

❖ The Technical Hurdle:

- Deleting a single user record from immutable, compressed columnar files (like Parquet) in a Data Lake is technically difficult. You typically have to rewrite the entire 1GB file just to remove 1KB of data.

❖ The Fix (Lakehouse):

- Utilize modern Table Formats (Apache Iceberg, Delta Lake) which support ACID DELETE transactions.
 - Copy-on-Write: The engine rewrites the file immediately without the deleted record (slower write, faster read).
 - Merge-on-Read: The engine writes a small "tombstone" file indicating the record is deleted. The reader merges this at query time (faster write, slower read).

❖ Crypto-Shredding

- An alternative pattern where you delete the encryption key associated with a specific user's data, rendering it unreadable without physically scrubbing the bytes.

Organizing for Speed: Partitioning

❖ Concept

- Partitioning physically divides your data into hierarchical folders based on the value of a specific column .

❖ Example

- If you partition by `Date`, your storage structure looks like: `/sales/date=2025-01-01/file.parquet`,
`/sales/date=2025-01-02/file.parquet`.

❖ Why it works (Pruning):

- When you run `SELECT * FROM sales WHERE date = '2025-01-01'`, the engine looks at the folder paths and immediately "prunes" (ignores) every folder that doesn't match `2025-01-01`. It reads only the relevant slice of data, which might be 1/365th of the total volume.

❖ Best Use Case: Low Cardinality columns.

- These are columns with relatively few unique values, such as `Date` (365 values/year), `Region` (5-50 values), or `Country`.

Organizing for Speed: Bucketing (Clustering)

❖ Concept

- Bucketing is a technique used when you need to organize data by a column with High Cardinality (millions of unique values), like `User_ID`.

❖ The Problem with Partitioning here

- If you partitioned by `User_ID`, you would create millions of tiny folders (one per user). The file system would collapse under the metadata load.

❖ The Solution

- You hash the `User_ID` into a fixed number of "buckets" (files). For example, if you choose 50 buckets, User A might go to `bucket_1`, User B to `bucket_49`, and so on.

❖ Why it works:

- It groups data together. If you join two tables on `User_ID` and both are bucketed by `User_ID`, the engine knows exactly which files correspond to each other, eliminating massive "shuffling" of data across the network.

The "Small Files" Anti-Pattern

- ❖ The most common performance killer in Data Lakes and modern Data Warehouses.
- ❖ The Problem: Over-Partitioning. This occurs when you partition data too granularly.
 - Example: A streaming job writes data every few seconds and partitions by Hour.
 - If the data volume is low, you might end up with thousands of partitions, each containing a tiny 1KB file.
- ❖ The Impact: Metadata Overhead
 - Distributed query engines utilize a central node that must "list" all the files before it can assign work to worker nodes.
 - Listing 1,000 files takes milliseconds. Listing 10 million files can take minutes or even crash the driver due to memory exhaustion. The query spends 90% of its time just finding the files and only 10% reading data.
- ❖ The Fix: Compaction
 - Compaction is a maintenance process that fixes this fragmentation. A background job scans the directory, identifies tiny files, reads them into memory, and rewrites them as a single file.
 - The next query only has to "list" one file header instead of thousands, restoring performance.

Summary: Modern Storage Architectures

❖ Strategic Decision Matrix

- Use Warehouse: If you need sub-second dashboards and your data is structured.
- Use Lakehouse: If you have massive volume, need flexibility, or use AI/ML.
- Use Mesh: If you are a large enterprise with many disconnected teams.

❖ The Convergence:

- The industry has shifted from the rigid Data Warehouse (high performance, high cost) and the unstructured Data Lake (low cost, high flexibility) to the Data Lakehouse.
 - By adding a metadata layer (Iceberg/Delta) over cheap object storage, Lakehouse gain ACID transactions, schema enforcement, and decoupled compute without the "warehouse tax".

❖ Optimization & Strategy

- Performance Tuning:
 - Speed relies on physical organization. Use Partitioning for low-cardinality dates and Bucketing for high-cardinality IDs. Always compact data to avoid the "Small Files" anti-pattern.
- Cost & Lifecycle:
 - Combat the "Write Once, Pay Forever" trap by implementing Lifecycle Policies (TTL) that move stale data to Cold/Archive tiers automatically.
- Decentralization:
 - For large enterprises, the Data Mesh pattern treats "data as a product," distributing ownership to domain teams while maintaining federated governance.

Further Reading & Resources

- ❖ Reis & Housley, Fundamentals of Data Engineering,
 - Chapters 2 and 6
 - <https://learning.oreilly.com/library/view/fundamentals-of-data/9781098108298/>
- ❖ Zhamak Dehghani, Data Mesh: Delivering Data-Driven Value at Scale
 - Chapters 1-3
 - <https://www.oreilly.com/library/view/data-mesh/9781492092384/>